Amazon AMI — 30 Important Interview Questions & Answers

1. What is an AMI?
Answer:
AMI stands for Amazon Machine Image. It is a template used to create EC2 instances with a predefined operating system, software, configuration, and settings. Instead of setting up a new server from the beginning every time, we can create an AMI and use it to launch new EC2 instances.
For example, suppose we have an EC2 server with Ubuntu, Java, application software, and required configurations. We can create an AMI from that server and use the AMI to launch another EC2 instance with the same setup.
Example:
Existing EC2
↓
Create AMI
↓
AMI
↓
Launch EC2
↓
New EC2 with same setup
Simple ga gurthupettuko:
AMI = Template for creating EC2 instances

---

2. Why do we use AMI?
Answer:
AMI is mainly used to create EC2 instances quickly with the same configuration. It saves the time required to install the operating system, applications, and other software again. AMIs are also useful for scaling and disaster recovery.
For example, if we already configured one EC2 server properly, we can create an AMI and use it to launch 10 more EC2 instances with the same setup.
Example:
Configured EC2
↓
AMI
↓
EC2 EC2 EC2 EC2
Simple ga gurthupettuko:
AMI = Create same EC2 setup quickly

---

3. What does an AMI contain?
Answer:
An AMI contains the information required to launch an EC2 instance. It includes the operating system, application software, and configuration stored in the associated storage. The exact contents depend on how the AMI was created.
For example, an AMI can contain Ubuntu, Java, Nginx, application files, and required configuration.
Example:
AMI
↓
Operating System
↓
Software
↓
Application
↓
Configuration
Simple ga gurthupettuko:
AMI = OS + Software + Configuration

---

4. What are the main components of an AMI?
Answer:
An AMI mainly contains a template for the root volume, block device mapping, and information about the instance architecture and launch configuration. The block device mapping tells EC2 which storage volumes should be attached when the instance starts.
For example, an AMI can define a root EBS volume and additional EBS volumes that should be attached to the new instance.
Example:
AMI
↓
Root Volume
↓
Additional Volumes
↓
EC2 Instance
Simple ga gurthupettuko:
AMI = EC2 launch information + Storage mapping

---

5. What is a custom AMI?
Answer:
A custom AMI is an AMI created by us from an existing EC2 instance or another supported source. It contains our own operating system setup, applications, and configurations. It is useful when we want to create multiple servers with the same environment.
For example, we configure one EC2 server with Java and our application and create a custom AMI from it.
Example:
Configured EC2
↓
Custom AMI
↓
New EC2 Instances
Simple ga gurthupettuko:
Custom AMI = Our own EC2 template

---

6. What is an AWS-provided AMI?
Answer:
AWS provides ready-made AMIs for common operating systems and environments. These AMIs help us launch an EC2 instance without manually installing the operating system. Examples include Amazon Linux and supported Ubuntu images.
For example, while launching an EC2 instance, we can select an Amazon Linux AMI and immediately create the server.
Example:
AWS AMI
↓
Amazon Linux
↓
Launch EC2
Simple ga gurthupettuko:
AWS AMI = Ready-made server template

---

7. What is the difference between AMI and EC2?
Answer:
An AMI is a template used to create an EC2 instance, while an EC2 instance is the actual running virtual server. We can create multiple EC2 instances from the same AMI. The AMI itself is not the running server.
For example, one AMI can be used to launch many EC2 instances.
Example:
AMI
/ | \
↓ ↓ ↓
EC2 EC2 EC2
Simple ga gurthupettuko:
AMI = Template
EC2 = Running server

---

8. How do we create an AMI from an EC2 instance?
Answer:
We can select an EC2 instance and choose the option to create an image. AWS creates an AMI based on the instance configuration and associated storage. We can then use that AMI to launch new instances.
For example, after completing application setup on EC2, we create an AMI so that the same setup can be reused.
Example:
EC2
↓
Create Image
↓
AMI
↓
Launch New EC2
Simple ga gurthupettuko:
EC2 → Create Image → AMI

---

9. What is an AMI ID?
Answer:
An AMI ID is the unique ID used to identify a particular AMI in an AWS Region. We use the AMI ID when launching an EC2 instance from that AMI. Different AMIs have different IDs.
For example, while launching EC2 through CLI or Terraform, we provide the required AMI ID.
Example:
AMI
↓
ami-xxxxxxxx
↓
Launch EC2
Simple ga gurthupettuko:
AMI ID = Unique ID of an AMI

10. Can we launch multiple EC2 instances from one AMI?
Answer:
Yes. We can use the same AMI to launch multiple EC2 instances. This is very useful when we need multiple servers with the same operating system and application setup. It is also commonly used with Auto Scaling Groups.
For example, one application AMI can be used to launch 5 EC2 instances.
Example:
AMI
↓

↓ ↓ ↓ ↓ ↓
EC2 EC2 EC2 EC2 EC2
Simple ga gurthupettuko:
One AMI → Many EC2

---

11. What is AMI sharing?
Answer:
AMI sharing allows us to make an AMI available to another AWS account. The other account can then use the shared AMI to launch EC2 instances, subject to the AMI and encryption permissions. This is useful when teams or accounts need to use the same server template.
For example, a company can create an application AMI in one AWS account and share it with another account.
Example:
Account A
↓
AMI
↓
Share
↓
Account B
↓
EC2
Simple ga gurthupettuko:
AMI Sharing = Share AMI with another account

---

12. Can we copy an AMI to another Region?
Answer:
Yes. We can copy an AMI from one AWS Region to another Region. This is useful for disaster recovery and for launching the same application in multiple Regions. The copied AMI becomes an AMI in the destination Region.
For example, we have an AMI in Mumbai and want the same application setup in Singapore.
Example:
Mumbai
↓
AMI
↓
Copy AMI
↓
Singapore
↓
AMI
Simple ga gurthupettuko:
AMI Copy = Use same image in another Region

---

13. What is AMI deregistration?
Answer:
Deregistering an AMI removes the AMI from the list of available AMIs in that Region. It does not automatically mean that all underlying snapshots are deleted. We need to manage associated snapshots separately.
For example, if an old AMI is no longer required, we can deregister it and clean up unused snapshots if appropriate.
Example:
Old AMI
↓
Deregister
↓
AMI no longer available for launch
Simple ga gurthupettuko:
Deregister = Remove AMI from available images

---

14. What is an AMI snapshot?
Answer:
For EBS-backed AMIs, AWS uses EBS snapshots as part of the image creation process. These snapshots contain the data needed for the EBS volumes used by the AMI. When a new instance is launched, volumes are created from these snapshots.
For example:

EC2
↓
AMI
↓
EBS Snapshot
↓
New EBS Volume
↓
New EC2
Simple ga gurthupettuko:
AMI + EBS Snapshot = Storage needed to create the instance

---

15. What is an EBS-backed AMI?
Answer:
An EBS-backed AMI uses Amazon EBS volumes for the instance's root storage. When we launch an instance from this AMI, the required EBS volumes are created and attached. Most modern EC2 instances commonly use EBS-backed storage.
For example, an AMI can contain an Ubuntu root volume stored as an EBS snapshot.
Example:
AMI
↓
EBS Snapshot
↓
EBS Volume
↓
EC2
Simple ga gurthupettuko:
EBS-backed AMI = AMI using EBS for storage

---

16. What is an AMI launch permission?
Answer:
AMI launch permissions control who can use an AMI to launch EC2 instances. An AMI can be private, shared with specific AWS accounts, or public depending on how it is configured. We should be careful when sharing custom AMIs because they may contain sensitive software or configuration.
For example, a company can keep its internal application AMI private and allow only selected accounts to use it.
Example:
Private AMI
↓
Allowed Account
↓
Launch EC2
Simple ga gurthupettuko:
Launch Permission = Who can use the AMI

---

17. What is a public AMI?
Answer:
A public AMI can be made available for other AWS users to use. Public images are commonly provided for operating systems or software products. Before using a public AMI, we should check that it comes from a trusted source.
For example, a software vendor can provide a public AMI containing its application.
Example:
Public AMI
↓
AWS Users
↓
Launch EC2
Simple ga gurthupettuko:
Public AMI = Available for public use

---

18. What is a private AMI?
Answer:
A private AMI is available only to the AWS account that owns it unless it is explicitly shared. It is commonly used for internal applications and company server configurations. Private AMIs provide better control over who can use the image.
For example, a production application AMI can be kept private inside the company's AWS account.
Example:
Private AMI
↓
Company Account
↓
EC2
Simple ga gurthupettuko:
Private AMI = Internal use

---

19. Can we encrypt AMI data?
Answer:
Yes. For EBS-backed AMIs, the underlying EBS snapshots can be encrypted using AWS KMS keys. When sharing or copying encrypted AMIs, the required KMS permissions must also be considered. Encryption helps protect stored data.
For example:
AMI
↓
Encrypted EBS Snapshot
↓
KMS Key
↓
Secure Storage
Simple ga gurthupettuko:
AMI Encryption = Protect AMI storage data

---

20. What is AMI lifecycle management?
Answer:
AMI lifecycle management means regularly creating, updating, and removing AMIs as application versions change. Old AMIs that are no longer required can be cleaned up along with their unused snapshots. This helps reduce storage cost and keeps the environment organized.
For example:
AMI v1 → Old
AMI v2 → Current
AMI v3 → New
Simple ga gurthupettuko:
AMI Lifecycle = Create → Use → Update → Clean up

---

21. How is AMI used with Auto Scaling?
Answer:
An Auto Scaling Group uses an AMI as part of its launch configuration or launch template. When new EC2 instances are required, the Auto Scaling Group uses that AMI to create them with the required setup. This makes automatic scaling much easier.
For example:
Auto Scaling Group
↓
Launch Template
↓
AMI
↓
New EC2 Instances
Simple ga gurthupettuko:
AMI + ASG = Automatically create same servers

---

22. What is the difference between AMI and Snapshot?
Answer:
An AMI is used to launch an EC2 instance, while a snapshot is mainly a point-in-time copy of an EBS volume. An AMI can use snapshots as part of its storage information. So they are related, but they are not the same thing.
For example:
Snapshot → Backup of EBS volume

AMI → Template to launch EC2
Simple ga gurthupettuko:
Snapshot = Volume copy
AMI = EC2 launch template

---

23. What is the difference between AMI and Backup?
Answer:
An AMI is mainly used to create EC2 instances with a particular setup. A backup is mainly used to recover data if something goes wrong. An AMI can help with recovery, but it should not automatically be treated as a complete backup strategy for every application.
For example, we can use backups for data recovery and AMIs for rebuilding application servers.
Example:
Backup → Recover Data

AMI → Rebuild EC2
Simple ga gurthupettuko:
Backup = Data recovery
AMI = Server creation

---

24. Why should we create a new AMI after application changes?

Answer:
If we make important changes to the operating system or application, we may create a new AMI so that future EC2 instances contain the latest setup. This gives us a clean and repeatable server image. It also helps Auto Scaling launch the correct application version.
For example:
Old AMI
↓
Application Update
↓
New AMI
↓
New EC2
Simple ga gurthupettuko:
New application version → New AMI

---

25. What happens when we launch an EC2 instance from an AMI?
Answer:
AWS uses the AMI information to create the required storage and launch the EC2 instance with the selected configuration. The instance gets the operating system and software included in the image. We then configure networking, security groups, instance type, and other launch settings as required.
For example:
AMI
↓
Select Instance Type
↓
Select Network
↓
Select Security Group
↓
Launch EC2
Simple ga gurthupettuko:
AMI provides the server image; launch settings provide the environment.

---

26. What is AMI versioning?
Answer:
AMI versioning means maintaining different AMIs for different application or configuration versions. It helps us know which image is being used in each environment. This is useful for testing, deployment, and rollback.
For example:
AMI-v1 → Application v1
AMI-v2 → Application v2
AMI-v3 → Application v3
Simple ga gurthupettuko:
AMI Version = Different server setup versions

---

27. Can we use the same AMI for development and production?
Answer:
Yes, if the same server setup is required, we can use the same AMI in different environments. However, environment-specific configuration such as secrets, endpoints, and environment variables should be handled separately. We should not store sensitive secrets directly inside an AMI.
For example:
AMI
/ \
↓ ↓
Dev EC2 Prod EC2
Simple ga gurthupettuko:
Same AMI → Dev + Test + Prod, when appropriate

---

28. What is the main advantage of AMI in real projects?
Answer:
The main advantage is repeatability. We can create a tested server image once and use it to launch many consistent EC2 instances. This reduces manual setup and configuration mistakes.
For example:
Tested AMI
↓
EC2-1
EC2-2
EC2-3
EC2-4
Simple ga gurthupettuko:
AMI = Same tested setup everywhere

---

29. How is AMI useful for disaster recovery?
Answer:
AMI can help us rebuild application servers quickly if the original server is lost. We can keep suitable AMIs available and launch new EC2 instances from them. For complete disaster recovery, we also need to consider application data, databases, networking, and other resources.
For example:
Original EC2
X
↓
Stored AMI
↓
New EC2
↓
Application Restored
Simple ga gurthupettuko:
AMI = Quickly rebuild server

---

30. What is the most important thing to remember about AMI?
Answer:
AMI is a reusable template for launching EC2 instances with a predefined operating system, software, and configuration. It helps us create servers quickly and consistently. AMIs are commonly used with Auto Scaling, deployments, testing, and disaster recovery.
For example, if we have one properly configured application server, we can create an AMI from it and use that AMI to launch multiple identical EC2 instances.
Example:
Configured EC2
↓
AMI
↓

↓ ↓ ↓
EC2 EC2 EC2
Simple ga gurthupettuko:
AMI = EC2 Server Template
